BUG BOUNTY FINDINGS

WOCS'hACK 2025

Event	WOCS'hACK 2025 (CTF-style bug bounty)
Scope	`*.3xploit.me` (organizer-built target)
Period	April 2025
Disclosure	Published with the organizers' authorization
Author	M3RICK

> **Context.** This was my first hands-on vulnerability research, on a dedicated environment built by the WOCS'hACK organizers. The three findings below were accepted during the event and are summarized here in my own words. No real third-party data was involved; the target infrastructure belonged to the event.

No.	Vulnerability	Type	CVSS 3.1	Status
1	SQL Injection via GET parameter	CWE-89	9.8 Critical	Accepted
2	Unrestricted file upload leading to RCE	CWE-434	9.6 Critical	Accepted
3	Source code disclosure via `php://filter`	CWE-260	7.5 High	Accepted

1 SQL Injection via GET parameter

Severity	Critical (CVSS 9.8, `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`)
Endpoint	`/association/list.php`
Parameter	`search` (GET)
Tooling	`sqlmap`

1.1 Description

The `search` GET parameter reached the SQL query without sanitization or parameterization, so a crafted value changed the structure of the query itself. I confirmed the injection and enumerated the database with `sqlmap`, recovering the schema.

1.2 Impact

Full read access to the database (user data and internal records), with write and availability impact depending on the privileges of the account the application uses.

1.3 Remediation

Use prepared statements / parameterized queries. User input must never be concatenated into SQL.

2 Unrestricted File Upload leading to RCE

Severity	Critical (CVSS 9.6, `AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H`)
Feature	Profile picture upload
Location	web-reachable `/uploads/`
Payload	PHP code carried inside an image (`.png`)

2.1 Description

The profile-picture upload accepted a file whose content was PHP code disguised as an image. The server validated neither the real file type nor the stored location, and placed the file under a web-reachable path. The uploaded file could then be requested directly and executed as PHP.

2.2 Impact

Potential remote code execution on the host.

2.3 Remediation

Validate the real content type (magic bytes, not the extension), store uploads outside the web root, serve them through a handler that never executes them, and randomize stored filenames.

3 Source Code Disclosure via php:

Severity	**High (CVSS 7.5, `AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N`)**
Endpoint	`/index.php`
Parameter	`page` (GET)

3.1 Description

The `page` GET parameter was passed to an include without restriction, which allowed a `php://filter` wrapper:

```
index.php?page=php://filter/convert.base64-encode/resource=config.php
```

This returned the base64-encoded source of `config.php`, exposing the database credentials stored in it.

3.2 Impact

Disclosure of backend source code and hardcoded secrets, usable to pivot to a full database compromise.

3.3 Remediation

Whitelist includable pages, reject stream wrappers in user input, and keep secrets out of source files (environment variables or a secrets manager).